Module Code and Title	**:**	WEB404 Secure Coding Practices
Programme	**:**	BE in Software Engineering
Credit	**:**	12
Module Tutor	**:**	Kamal Acharya/Douglas Sim/Ashis Chettri
Module Coordinator	**:**	Douglas Sim

General Objective:
This module aims to provide a comprehensive understanding of web assessment methodologies through testing and checking vulnerabilities of a web application. This introductory course is designed to equip students with the essential knowledge and skills required to assess and secure a web application. The topics covered include server-side attacks such as SQLi, Directory Traversal, and Command Injection, and client-side attacks such as CSS, CSRF, CORS, and Dom-Based vulnerabilities. By the end of this module, students will have a solid foundation in web application assessment that will serve as a basis for securing their web applications.

Learning outcomes:
On completion of the module, students will be able to:

1. Describe the key components of internet architecture, common internet protocols, and web security assessment tools.
2. Explain the mechanisms behind common web attacks, including SQL injection, XSS, and CSRF, and describe their potential impacts on web applications.
3. Use web security assessment tools like Burp Suite and OWASP Zap to identify vulnerabilities in web applications and generate comprehensive security reports.
4. Examine web applications to identify and categorize various types of server-side vulnerabilities, including SQL injection, directory traversal, and command injection.
5. Assess the effectiveness of different security measures in preventing server-side and client-side attacks, and propose improvements where necessary.
6. Develop secure web applications that implement robust defences against common attack vectors, including proper input validation, output encoding, and access controls.
7. Implement secure coding practices and preventive measures to mitigate advanced web attacks, such as insecure deserialization, server-side template injection, and JWT attacks.
8. Construct secure API architectures using GraphQL or RESTful principles, incorporating measures to prevent common API vulnerabilities and ensure robust access control.

Learning and teaching approach:

Type	**Approach**	**Hours per Week**	**Total credit hours**
Contact	Lecture	3	**75**
	Practical	2	

Independent Study	Assignment	2	**45**
	Consultation	1	
Total		**8**	**120**

Assessment Approach:
Assessment components consist of **Continuous Assessment (CA) theory - 50%**, and **Continuous Assessment (CA) Practical - 50%**, The CA theory consists of a mid-term test, class test, quiz and presentation. The CA practical will include practical assignments each weighing 10%.

A. **Mid-term Test: (15%)**
Students will take a written test of 2-hour duration covering topics up to the middle of the semester.

B. **Class Test: (15%)**
Students will take a written test of 1 hour duration covering topics after the mid-term test in the 10th week.

A. **Quiz (10%)**
Students will take a class quiz of 1 hour duration covering topics after the mid-term test in the 12th week.

C. **Practical Assignments (50%)**
Students are required to complete five practical assignments throughout the semester. Each assignment will build upon the concepts covered in the lectures and practical sessions.

Each assignment shall be assessed out of 10 marks as follows:

Practical Assignment: (10%)

Students have to complete the assignment as per the requirements provided by the module tutor and submit their work to the designated online version control system allocated by the HoD, SWE. Each assignment will be evaluated using the following criteria:

3.25 Technical Accuracy & Methodology
3.25 Application of concepts
2 Presentation and elaboration
1.5 Timely Submission and Language

D. **Presentation (10%)**

The student has to make one 10-minute presentation on one of the assignments provided by the module tutor. The presentations will be assessed based on the following criteria:

2 Content
3 Delivery
3 Answer to questions
2 Presentation aids

Overview of the assessment approaches and weighting

Areas of Assessment	**Quantity**	**Weighting (%)**
A. Mid Term Test	1	15
B. Class Test	1	15
C. Assignments	4	50
D. Quiz	1	10
E. Presentation	1	10
Total		100

Pre-requisites: WEB101 Web Application Fundamentals, WEB102, Server Aplication Fundamentals

Subject matter:
Unit I: Introduction to Web Security
1.1 How the Internet works
- 1.1.1 Internet architecture
- 1.1.2 Client-server model
- 1.1.3 Socket programming

1.2 Internet Protocols
- 1.2.1 HTTP 1.0 and 1.1
- 1.2.2 HTTP/2
- 1.2.3 HTTP/3
- 1.2.4 IP (Internet Protocol)
- 1.2.5 HTTP headers
- 1.2.6 HTTP body

1.3 Web Security Assessment Tools
- 1.3.1 Burp Suite
- 1.3.2 OWASP ZAP (Zed Attack Proxy)
- 1.3.3 Nikto

Unit II: Server Side Attacks

2.1 SQL Injection
- 2.1.1 Types of SQL injection
- 2.1.2 Blind SQL injection
- 2.1.3 SQL injection prevention

2.2 Directory Traversal
- 2.2.1 Path manipulation techniques
- 2.2.2 Preventing directory traversal attacks

2.3 File Upload Vulnerability
- 2.3.1 Malicious file uploads
- 2.3.2 File type and content validation
- 2.3.3 Secure file handling practices

2.4 Information Disclosure
- 2.4.1 Sensitive data exposure
- 2.4.2 Error message handling
- 2.4.3 Preventing information leakage

2.5 Command Injection
- 2.5.1 OS command injection techniques
- 2.5.2 Command injection prevention

2.6 Authentication
- 2.6.1 Authentication bypass techniques
- 2.6.2 Brute force attacks
- 2.6.3 Secure authentication practices

2.7 Preventive measures against server-side attacks
- 2.7.1 Input validation and sanitization
- 2.7.2 Parameterized queries
- 2.7.3 Least privilege principle
- 2.7.4 Web application firewalls (WAF)

Unit III: Advanced Server Side Attacks

3.1 Business Logic Vulnerabilities
- 3.1.1 Identifying business logic flaws
- 3.1.2 Testing for business logic vulnerabilities
- 3.1.3 Mitigating business logic attacks

3.2 Access Control
- 3.2.1 Vertical privilege escalation
- 3.2.2 Horizontal privilege escalation
- 3.2.3 Implementing proper access controls

3.3 Race Conditions
- 3.3.1 Time-of-check to time-of-use (TOCTOU) vulnerabilities
- 3.3.2 Concurrency issues
- 3.3.3 Preventing race conditions

3.4 Server-Side Request Forgery (SSRF)
- 3.4.1 SSRF attack techniques
- 3.4.2 SSRF impact and exploitation
- 3.4.3 SSRF prevention strategies

3.5 XXE Injection
    3.5.1 XML external entity attacks
    3.5.2 XXE exploitation techniques
    3.5.3 Preventing XXE vulnerabilities
3.6 Preventive measures against Access Control, SSRF & XXE attacks
    3.6.1 Implementing proper authorization mechanisms
    3.6.2 Whitelisting allowed resources and URLs
    3.6.3 Disabling XML external entity processing

Unit IV: Client Side Attacks
4.1 Cross-site Scripting (XSS)
    4.1.1 Reflected XSS
    4.1.2 Stored XSS
    4.1.3 DOM-based XSS
    4.1.4 XSS prevention techniques
4.2 Cross-site request forgery (CSRF)
    4.2.1 CSRF attack mechanisms
    4.2.2 CSRF tokens
    4.2.3 CSRF prevention strategies
4.3 Cross-origin resource sharing (CORS)
    4.3.1 Same-origin policy
    4.3.2 CORS configuration
    4.3.3 Secure CORS implementation
4.4 Clickjacking
    4.4.1 UI redressing attacks
    4.4.2 Frame busting techniques
    4.4.3 X-Frame-Options header
4.5 DOM-based vulnerabilities
    4.5.1 DOM manipulation attacks
    4.5.2 Client-side input validation issues
    4.5.3 Preventing DOM-based vulnerabilities
4.6 WebSockets
    4.6.1 WebSocket security risks
    4.6.2 Secure WebSocket implementation
    4.6.3 WebSocket vulnerability testing
4.7 Preventive Measures Against Client Side Attacks
    4.7.1 Content Security Policy (CSP)
    4.7.2 Input validation and output encoding
    4.7.3 Secure cookie attributes
    4.7.4 Browser security headers

Unit V: Advanced Web Attacks
5.1 Insecure Deserialization
    5.1.1 Serialization and deserialization processes
    5.1.2 Exploiting insecure deserialization
    5.1.3 Secure deserialization practices

5.2 GraphQL API vulnerabilities
    5.2.1 GraphQL injection
    5.2.2 Batching attacks
    5.2.3 Securing GraphQL APIs
5.3 Server-side template injection
    5.3.1 Template engine vulnerabilities
    5.3.2 SSTI attack techniques
    5.3.3 Preventing template injection
5.4 Web cache poisoning
    5.4.1 Cache behavior exploitation
    5.4.2 Cache poisoning attack vectors
    5.4.3 Mitigating cache poisoning risks
5.5 HTTP host header attacks
    5.5.1 Host header injection
    5.5.2 Password reset poisoning
    5.5.3 Protecting against host header attacks
5.6 HTTP request smuggling
    5.6.1 CL.TE and TE.CL vulnerabilities
    5.6.2 Request smuggling attack techniques
    5.6.3 Preventing HTTP request smuggling
5.7 OAuth authentication
    5.7.1 OAuth 2.0 flow vulnerabilities
    5.7.2 Open redirector exploitation
    5.7.3 Implementing secure OAuth
5.8 JWT attacks
    5.8.1 JWT structure and vulnerabilities
    5.8.2 JWT signature bypasses
    5.8.3 Secure JWT implementation
5.9 Prototype pollution
    5.9.1 JavaScript prototype chain exploitation
    5.9.2 Client-side prototype pollution
    5.9.3 Preventing prototype pollution attacks

List of Practical(s):

1. Use Nikto to conduct a vulnerability assessment on a sample website and interpret the findings.
2. Create a command-line interface in a web application and implement safeguards against command injection.
3. Develop a file upload feature and implement security measures to prevent malicious file uploads.
4. Configure and test a Web Application Firewall (WAF) to protect against common server-side attacks.
5. Set up a vulnerable web application and perform various types of SQL injection attacks.
6. Create a CSRF vulnerable form and then implement CSRF tokens to prevent attacks.
7. Develop a JWT-based authentication system, demonstrate common JWT attacks, and implement secure JWT handling.

Reading list:

Essential Reading

Hoffman, A. (2024). *Web Application Security: Exploitation and Countermeasures for Modern Web Applications*.

McDonald, M. (2020). *Web security for Developers: Real Threats, Practical Defense*. No Starch Press.

Stuttard, D., & Pinto, M. (2011). *The web application Hacker's handbook: Finding and Exploiting Security Flaws*. John Wiley & Sons.